

Entendendo o Zero Knowledge Proof

828561 - Antonio Adolfo Geraldino Bernardo

820562 - Ayla Yorinori Nishino

834418 - Carolina Megumi Nogai

834437 - Cauã Borges Faria

813641 - Eduardo Sanzovo Quaggi

824391 - Gabriel Somensi Duarte

824388 - Laura Pessine Teixeira

834416 - Rafaela Silva Ruis

834417 - Rafael Yugo Hashimoto

821626 - Ronan Antonio Pereira Junior

830916 - Yuki Hiroji Oyama



? Problema

Imagine que eu conheço uma senha. Como posso provar para alguém que realmente conheço essa senha sem contar qual é a senha?

Zero- Knowledge Proof (ZKP)

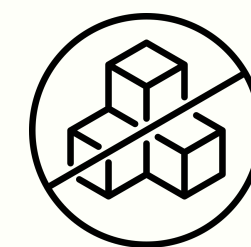
Protocolos criptográficos que permitem que uma parte (o provador) prove a outra parte (o verificador) que conhece uma informação específica sem revelar a própria informação



Completude



Solidez



Conhecimento Zero

Em suma, um protocolo que permite a uma pessoa provar que conhece uma informação secreta sem revelar essa informação.

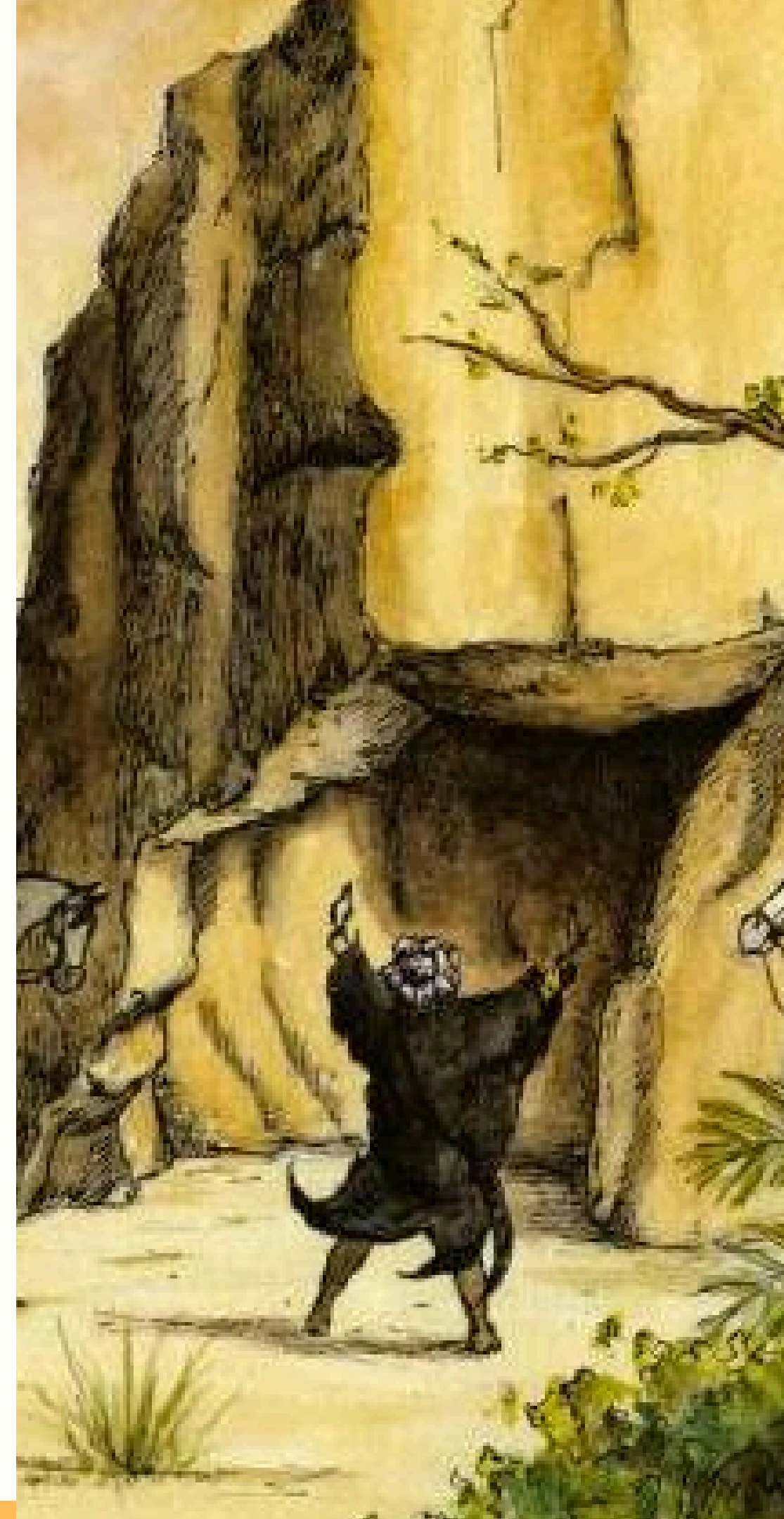
Analogia da caverna



Bob



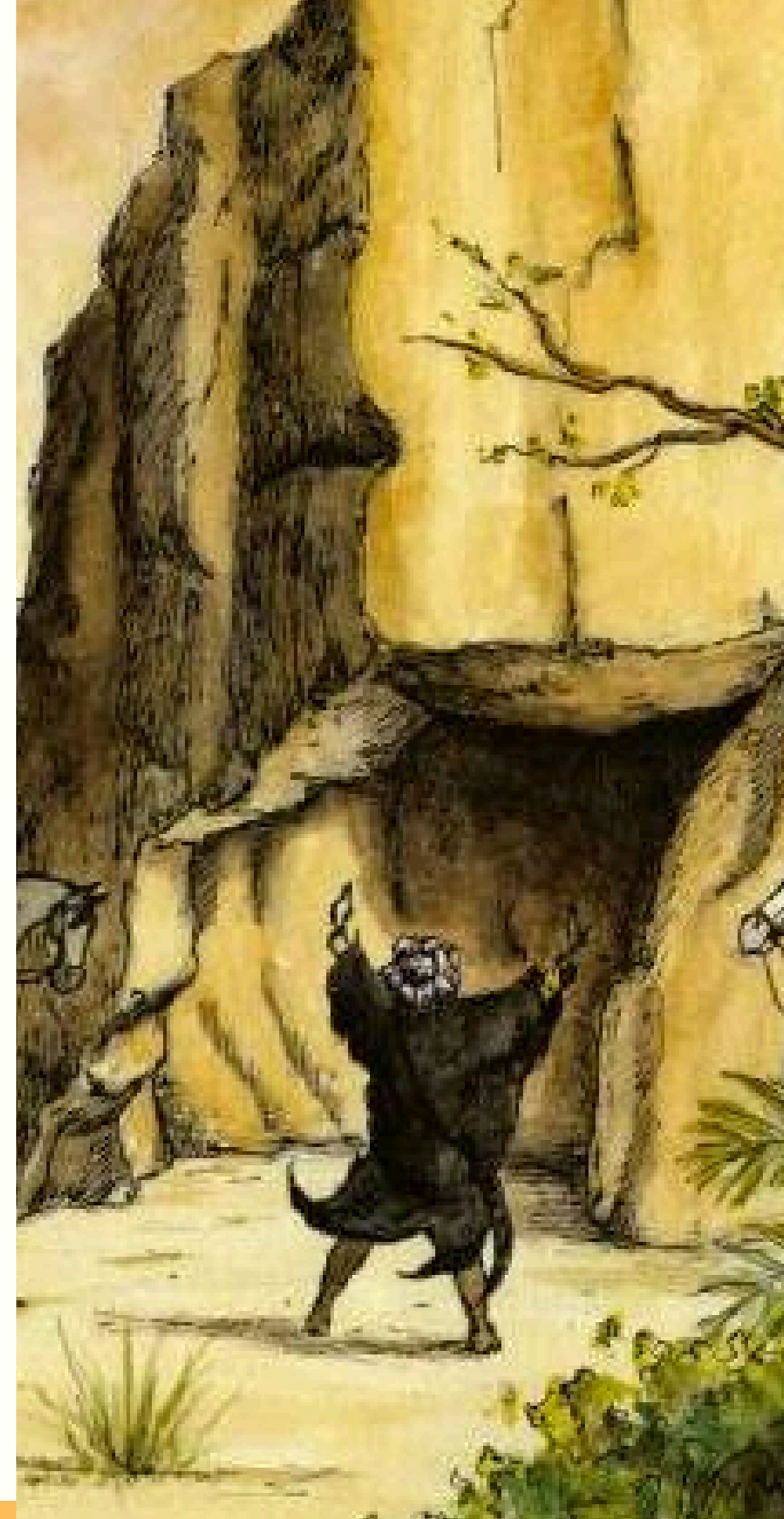
Ana



Probabilidade de fraude

$$p(\textit{enganar}) = (1/2)^n$$

Isso explica por que o desafio precisa ser **imprevisível**.



Mas, e matematicamente?

Entra em ação, então, a **criptografia**. No CTF, temos:

$$y = g^x \pmod{p}$$

x = a chave privada (o segredo)

g = o gerador do grupo multiplicativo

p = um número primo grande (módulo principal)

y = a chave pública



Mas, e matematicamente?

$$y = g^x \pmod{p}$$

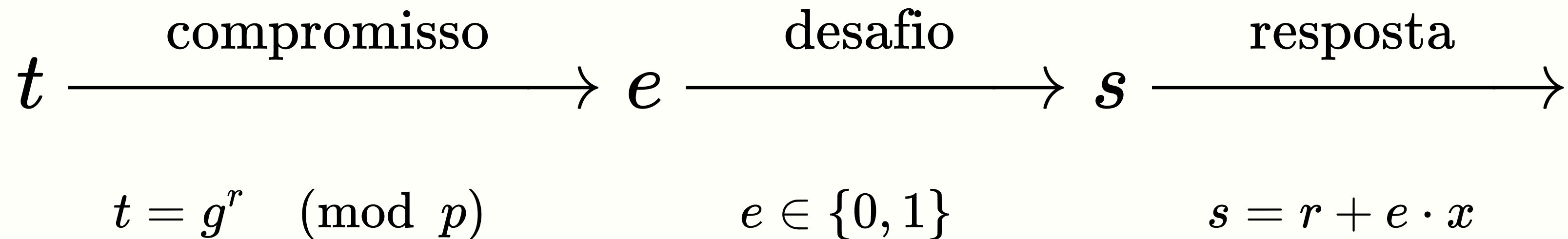
O **servidor** conhece **g**, **p** e **y**, mas não conhece **x**.

O **usuário** conhece **x**.

Descobrir **x** é um *problema computacionalmente difícil*.



Como deveria funcionar?



O compromisso (t) deve ser enviado **antes** do desafio (e)

Onde esteve a falha?

A vulnerabilidade no servidor do CTF

Ordem no servidor

$$e \longrightarrow t \longrightarrow s$$

Ordem correta

$$t \longrightarrow e \longrightarrow s$$

test_a $\implies e = 0$ (desafio pré-informado)

test_b $\implies e = 1$ (permite novo $t_1 \neq t_0$)

$$e \in 0, 1$$

O servidor revelou o desafio **e antes** do compromisso, então houve quebra de solidez.

solution.py

O Exploit

Transcrições Forjadas (sem o segredo x):

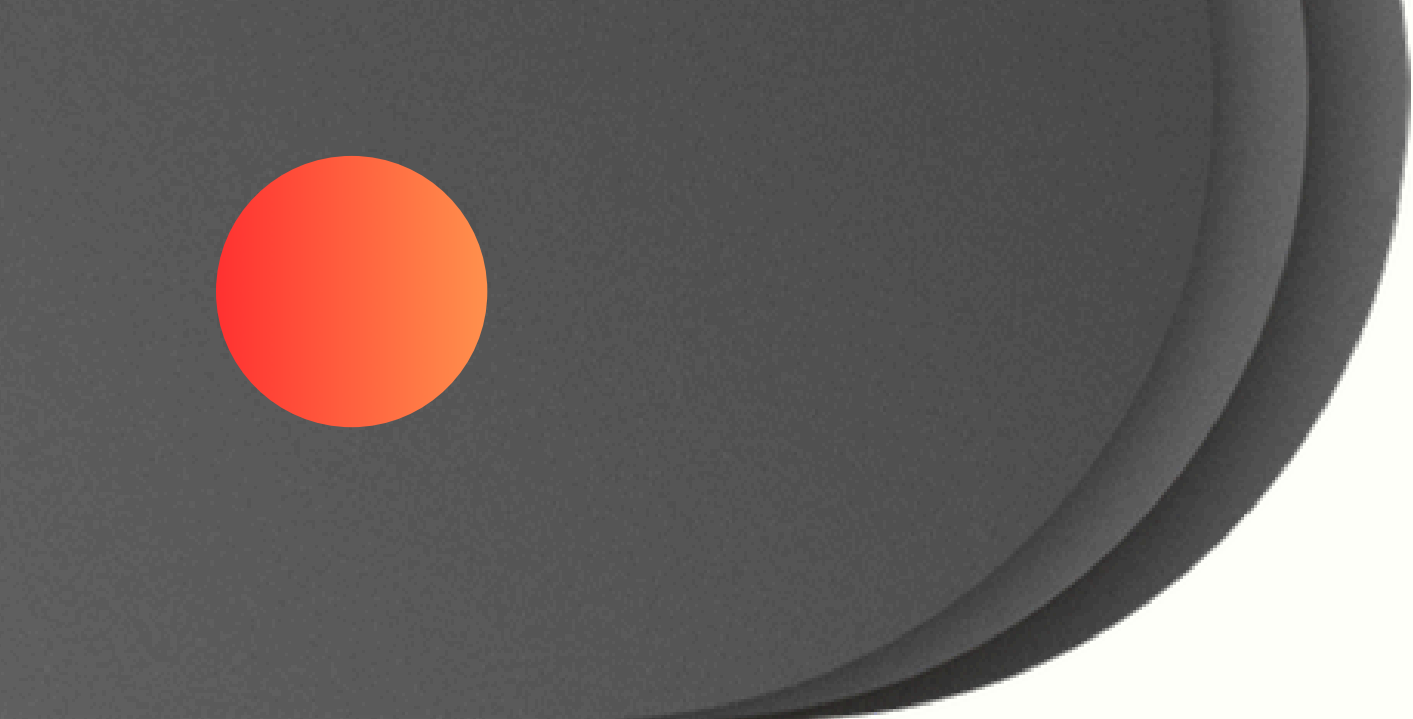
$$\text{Para } e = 0 \implies (t_0, s_0) = (1, 0) \implies g^0 \equiv 1 \pmod{p}$$

$$\text{Para } e = 1 \implies (t_1, s_1) = (y^{-1} \pmod{p}, 0) \implies g^0 \equiv y^{-1} \cdot y \equiv 1 \pmod{p}$$

Conclusão:

Matemática $(y = g^x \pmod{p}) \implies$ Inviolável

Ordem de Execução $(e \rightarrow t \rightarrow s) \implies$ Vulnerável



Conclusão

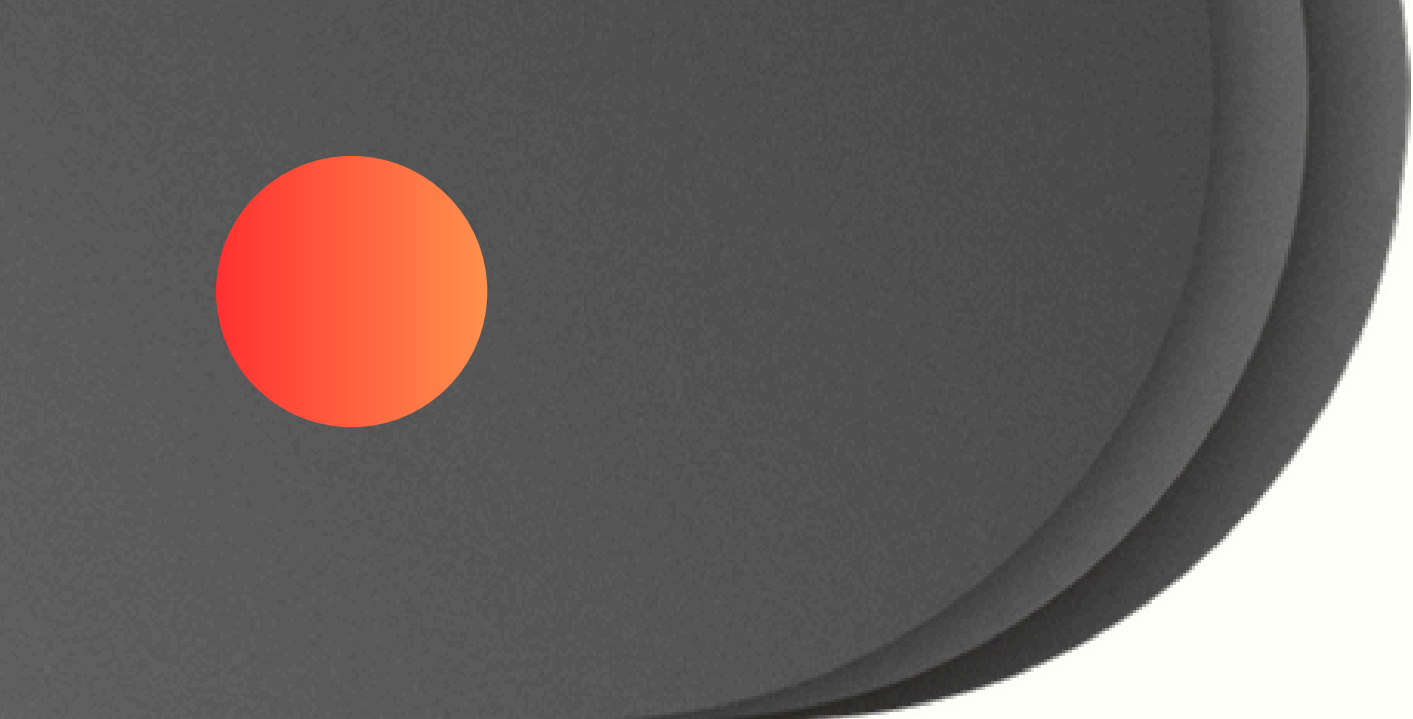
Consequência

Ao informar a pergunta **antes** de pedir o compromisso, o servidor permite que o atacante **forje** a matemática do teste sem possuir o segredo x .

Relação com OR-Proofs

O erro do servidor foi permitir duas simulações **desconectadas** sem amarrá-las matematicamente a um desafio global único.

A regra é enviar t antes de escolher e (obrigatoriedade do compromisso único)!



Demonstração

